Group Assignment

Subject : Information Systems Agile Project Management with DevOps

Class : IS 2026

Duration : Week 6

Lecturer : Eka Parima Saragih, S.Kom., M.Kom.

"Move beyond 'it works on my laptop.' Make it reproducible for another person."

Core Challenge: *Use a project that your group has already finished or can run successfully. Don't build a new app or add new features. Your goal is to package your existing app with Docker, prove that its main function still works, and set up a clear setup guide so anyone can run it from a fresh clone.*

1. Assignment Overview

In this assignment, your group will take an existing project and set up a Docker runtime for it. Focus on packaging, configuration, making sure it works, and writing clear instructions for handover, rather than coding new features.

Important Scope Rule: *You don't need to write a new app or build new features. Small config changes are totally fine if needed to get things running in a container, like binding to 0.0.0.0, reading environment variables, or pointing connections to Compose service names instead of localhost.*

1.1 Working Arrangement

- Form a group of up to max 3 students. Work together as a team to complete the assignment.
- Use one shared GitHub repo. Every member needs to use their own account and contribute with clear commits or pull requests.
- Divide up practical tasks, but make sure everyone in the group understands the final Docker workflow.
- Pick one team member to act as the handover tester to try running the instructions from a fresh clone or clean folder.

1.2 Allowed Project Types

You can use a web app, REST API, dashboard, mobile application, internal info system, or any project that clearly shows a runtime and at least **three** core working features. Just make sure the project already works before starting Dockerization.

2. Assignment Scenario

Imagine your team needs to hand over an existing project to another student. Right now, the app depends on manual local setup and might break on a different computer. Your group needs to build a Docker setup that packages everything needed and write a guide on how to build, start, check, inspect, and stop the app.

A good submission should cover all four levels below:

Level	Required evidence
1. Build	A Docker image can be created directly from the repository.
2. Run	The container or Compose services start up smoothly with no errors.
3. Verify	The app's main function works properly in a browser or API client.
4. Reproduce	Another student can easily repeat the steps from a fresh clone using the README.

3. Step-by-Step Instructions

Before You Begin: Install and start Docker Desktop (or your approved Docker setup). Make sure the engine is running by trying 'docker version' in your terminal. If your app uses multiple services, check that 'docker compose version' works too.

Step 1: Establish the Working Baseline

Run the project normally without Docker first. This helps make sure any bugs are fixed before you start adding Docker into the mix.

1. Clone or open your group's repository.
2. Follow your standard setup steps and start the app as usual.
3. Test out one main feature like logging in, loading a dashboard, or sending an API request.
4. Note down your runtime details, start command, port, env variables, database, and any connected services.
5. Keep baseline screenshots or notes for proof (just obscure any real secret keys or passwords!).

Baseline item	Group response
Project name and repository URL	
Main user or purpose	
Technology stack	
Normal start command	
Application port	
Required environment variables	
Supporting services	
Core function selected for verification	

Step 2: Create a Dockerization Branch

Keep your main working code safe! Create a new git branch for this assignment instead of editing main directly.

Example:

```
git checkout main
git pull
git checkout -b chore/dockerize-project
```

- Name your branch something descriptive like 'chore/dockerize-project' or 'feature/docker-setup'.

- Commit your Docker changes step-by-step with helpful messages.
- Open a pull request once your setup is ready for review.

Step 3: Analyze the Runtime

Before writing your Dockerfile, break down what the app needs to start. Don't just copy a generic Dockerfile from online without tailoring it to your project.

Question	What to identify
Base runtime	Examples: Node.js, Python, Java, PHP, .NET, Nginx.
Dependencies	The file and command used to install packages.
Build step	Whether the project must be compiled or bundled before it starts.
Start command	The command that keeps the application process running.
Container port	The port used inside the container.
Configuration	Environment variables and safe default values.
Data dependency	Database, cache, queue, local file storage, or external API.
Health check	A page or endpoint that confirms the application is ready.

Step 4: Create the Dockerfile

Create a file named Dockerfile (no extension) in your root folder. Adjust the commands to match your tech stack.

Generic Dockerfile Pattern: Pick a base image, set a working directory, copy dependency files, install packages, copy the rest of your app, expose the port, and set the start command. Replace placeholder values with your project settings.
Example:

```
FROM <appropriate-runtime-image>
WORKDIR /app

# Copy dependency definitions first
COPY <dependency-files> ./
RUN <install-dependencies-command>

# Copy the application files
COPY . .

# Build only if your project requires it
RUN <build-command>

EXPOSE <container-port>
CMD ["<executable>", "<argument-1>", "<argument-2>"]
```

Quick checklist for your Dockerfile:

- Did you pick the right base image and version for your stack?
- Are packages installed using the right manifest files and commands?
- Does the app bind to 0.0.0.0 inside the container if needed?
- Does the CMD or ENTRYPOINT command keep the process active?
- Are secret tokens and keys excluded from the image build?

Step 5: Create .dockerignore

Create a .dockerignore file in your main folder. Exclude local junk, node_modules, virtualenvs, cache files, build outputs, and sensitive secrets.

Security Rule: Never commit .env files with real secrets! Include a .env.example file with dummy default values instead. If you accidentally commit a real secret, clean it up right away.

Step 6: Build and Tag the Image

Build your image with a clear, lowercase name and tag:

```
docker build -t <group-name>/<project-name>:week6 .
```

Example command:

```
docker build -t group03/campushub:week6 .
```

- Keep track of the exact build command you used.
- Save output showing a successful build.
- Verify the image exists by running 'docker image ls'.
- If the build fails, check the first error message to see what went wrong.

Step 7: Run the Application

Choose the run option that matches your setup.

Option A: Single-service application

```
docker run -d \
  --name <container-name> \
  -p <host-port>:<container-port> \
  --env-file .env \
  <group-name>/<project-name>:week6
```

Example command:

```
docker run -d --name campushub-app -p 8080:3000 --env-file .env group03/campushub:week6
```

Option B: Multi-service application

If your project needs a database, cache, or extra services, set up a compose.yaml file. Only include services needed to run the project locally, and don't put hardcoded passwords in Compose files.

```
services:
  app:
    build: .
    ports:
      - "<host-port>:<container-port>"
    env_file:
      - .env
    depends_on:
      - db

  db:
    image: <approved-database-image>
    environment:
      <database-variable>: <safe-development-value>
    volumes:
      - app-data:/var/lib/<database-data-directory>

volumes:
  app-data:
```

```
docker compose up --build -d
docker compose ps
docker compose logs
```

Architecture Rule: Docker Compose is required if your app needs more than one service to run. It's optional for simple single-service apps.

Step 8: Inspect and Troubleshoot

Use Docker commands to diagnose issues instead of blindly rebuilding when something breaks.

Purpose	Command
List running containers	docker ps
List all containers	docker ps -a
Read container logs	docker logs <container-name>
Follow logs continuously	docker logs -f <container-name>
Inspect configuration	docker inspect <container-name>
Open a shell when available	docker exec -it <container-name> sh
View Compose status	docker compose ps
Read Compose logs	docker compose logs
Stop and remove Compose services	docker compose down

Step 9: Verify the Core Function

Building the image isn't enough; test that the main app features still work inside the container. Log at least three checks.

Check ID	Verification action	Expected result	Actual result / evidence
V-01	Container or services are running	Required service status is running or healthy	
V-02	Open health page or endpoint	Expected response is returned	
V-03	Perform one core user function	Function completes and correct data or output appears	
V-04 (optional)	Restart the container or Compose stack	Application starts again successfully	

Step 10: Prepare the README

Update README.md so another student can clone and run your project without needing extra help. Make sure it has these sections:

- Project overview and main feature.
- Prerequisites and Docker version tested.
- How to clone the repository and configure env variables.
- Required env variable names via .env.example (no real secrets!).
- Commands to build or start with Docker Compose.
- App URL, host port, and container port info.

- How to verify the app works and expected outputs.
- Commands to view logs and status.
- How to stop and clean up containers.
- Troubleshooting notes and known limitations.

Step 11: Handover Test

Have a member from another group to test the handover instructions (You choose them by yourself). Try testing from a fresh folder or a different computer if possible. And create the handover report + evidence for handover result. The evidence will be created by that other group that tested your handover instructions.

- Clone the repo from your group's branch.
- Set up your local .env file from .env.example.
- Follow only the steps written in the README file.
- Build and launch the containers.
- Perform the verification checks.
- Stop the app using the documented stop command.
- Note any confusing steps or errors so you can fix up the README.

Handover record	**Response**
Verifier name	
Device / operating system	
Fresh clone completed?	Yes / No
Image build successful?	Yes / No
Application start successful?	Yes / No
Core function verified?	Yes / No
Problems found and corrections made	

Step 12: Review, Merge, and Tag

- Push your Docker branch to GitHub.
- Open a PR and link it to your assignment task (you are allowed to integrate it with Jira).
- Ask a friend to review the Dockerfile, .dockerignore, Compose file, README, and evidence (this has to be someone from other group), after that put the evidence for handover result of them reviewing your docker components in a report (the pass/fail check and testing)
- Address feedback and make sure everything builds and runs cleanly.
- Merge into main branch using your team's workflow.
- Create a release tag like 'week6-docker-v1' if required.

4. Required Deliverables

Submit one combined submission package for your group. Make it in a zip along with the full report. Print ONLY the report file and put it in the respective lecturer’s desk.

No.	Deliverable	Minimum requirement
1	GitHub repository	Accessible repository with identifiable member contributions and final Docker configuration.
2	Docker configuration	Dockerfile, .dockerignore, and compose.yaml when more than one service is required.
3	README	Complete build, run, verify, logs, stop, and troubleshooting instructions.
4	Evidence report	One PDF using the evidence structure in Section 5.
5	Demonstration video	Maximum 20 minutes covering the projects information and then the docker setup.
6	Handover verification	Named verifier and result from a fresh clone or clean folder.
7	Individual contribution	Short contribution statement for every member.

Do Not Submit: Do not upload node_modules, virtual environments, build outputs, real database dumps, .env files with secrets, private keys, or exported Docker images unless specifically requested.

4.1 Demonstration Video Sequence

- Introduce your app and the main core feature you selected.
- Show key files in your repo without exposing any secret keys.
- Run the build command.
- Start the container or Compose stack.
- Show container status and sample log output.
- Open the app in your browser and perform the core feature live.
- Stop the container or stack using your documented stop command.
- State who carried out the fresh handover test.

5. Evidence Report Structure

Prepare a clear PDF report using this outline. Keep the evidence easy to read and linked to your repo version.

Section	Required content
A. Group identity	Group number, member names, student IDs, project title, repository URL, video URL.
B. Project baseline	Technology stack, normal start command, port, dependencies, core function, baseline evidence.
C. Container design	Image name and tag, base image, container port, environment variables, services, volume or network decisions.
D. Build evidence	Build command, successful result, image listing, and short explanation.
E. Runtime evidence	Run or Compose command, service status, logs, host URL, and port mapping.
F. Functional verification	At least three checks with expected result, actual result, and evidence.

Section	Required content
G. Handover result	Verifier, environment, exact steps followed, result, issues found, corrections made.
H. Troubleshooting reflection	One real problem, root cause, diagnostic evidence, solution, and lesson learned.
I. Contributions	Specific contribution and understanding statement for each member.

5.1 Individual Contribution Table

Member / Student ID	Specific contribution	Commit or PR evidence	Concept the member can explain

6. Submission Guidelines

- Submit the zip of your files in the given GDrive link (provided in Discord).
- Upload your evidence report as a PDF and provide links to your GitHub repo and demo video.
- Use file name format: W6_Group##_ProjectName.pdf.
- Keep your repository public or accessible.

Academic Integrity: You're welcome to read docs and examples online, but your Docker setup must be written for your own project and every group member must understand how it works.

7. Quick Troubleshooting Guide (just FYI!)

Symptom	***Check first***
Build cannot find a file	*Confirm the build context, Dockerfile location, COPY path, and .dockerignore rules.*
Dependency installation fails	*Check runtime version, package-lock or equivalent file, network access, and package-manager command.*
Container exits immediately	*Run docker ps -a and read docker logs; verify CMD or ENTRYPOINT.*
App runs but browser cannot connect	*Check -p host:container mapping and confirm the app listens on 0.0.0.0.*
Environment variable missing	*Confirm .env exists locally, variable names match, and --env-file or Compose env_file is used.*
App cannot reach database	*In Compose, use the database service name rather than localhost; check credentials and readiness.*

Symptom	***Check first***
Changes do not appear	*Rebuild the image, verify the correct tag and container, then remove the old container if necessary.*
Permission or file-write error	*Check container user, mounted directory permissions, and whether persistence is actually required.*

Good luck!